

# Report

dcs076

接下來的報告我會分成五個部分做說明，分別為輸入加速、GCD 找法設計、FSM 的應用、block diagram、心得。

## 1. 輸入加速

```
if (in_valid && slot_num < 3) begin
    state <= IDLE;
    input_num <= input_num + 1;
    if (in_data % 2 == 0) begin
        if (even_num == 1) begin
            array_B[slot_num] <= tmp_even + in_data;
            slot_num <= slot_num + 1;
            tmp_even <= 5'b0;
            even_num <= 0;
        end
        else begin
            tmp_even <= tmp_even + in_data;
            even_num <= even_num + 1;
        end
    end
    else begin
        if (odd_num == 1) begin
            array_B[slot_num] <= tmp_odd + in_data;
            slot_num <= slot_num + 1;
            tmp_odd <= 5'b0;
            odd_num <= 0;
        end
        else begin
            tmp_odd <= tmp_odd + in_data;
            odd_num <= odd_num + 1;
        end
    end
end
end
else if (in_valid && input_num < 7)
begin
    input_num <= input_num + 1;
end
end
```

為了讓 cycles 減至 0 和避免 GCD 最後計算結果為 1，我設計這一套算法，每一次讀入數字後會先判斷奇偶性並累加讀入的 input 數(input\_num)，判斷完之後就會進入判斷目前累積奇偶數有幾個(even\_num、odd\_num)，如果已經有兩個就直接放入 array\_B，反之，未滿的話就放入暫存區 (tmp\_even、tmp\_odd)，直到 array\_B 被填滿為止，如果在 input 還未傳完之前就填滿的就無條件增加 input\_num，以便判斷 output 要在什麼時候輸出，透過這方法可以將輸入在最後一個 input 的半 clock 就開始輸出。

## 2. GCD 找法設計

```
// 計算除數 2 的最大整除次數
temp_B0 = array_B[0];
temp_B1 = array_B[1];
temp_B2 = array_B[2];
if (temp_B0 % 2 == 0 && temp_B1 % 2 == 0 && temp_B2 % 2 == 0) begin
    temp_B0 = temp_B0 / 2;
    temp_B1 = temp_B1 / 2;
    temp_B2 = temp_B2 / 2;
    count_2 = count_2 + 1;
    if (temp_B0 % 2 == 0 && temp_B1 % 2 == 0 && temp_B2 % 2 == 0) begin
        temp_B0 = temp_B0 / 2;
        temp_B1 = temp_B1 / 2;
        temp_B2 = temp_B2 / 2;
        count_2 = count_2 + 1;
        if (temp_B0 % 2 == 0 && temp_B1 % 2 == 0 && temp_B2 % 2 == 0) begin
            temp_B0 = temp_B0 / 2;
            temp_B1 = temp_B1 / 2;
            temp_B2 = temp_B2 / 2;
            count_2 = count_2 + 1;
            if (temp_B0 % 2 == 0 && temp_B1 % 2 == 0 && temp_B2 % 2 == 0) begin
                temp_B0 = temp_B0 / 2;
                temp_B1 = temp_B1 / 2;
                temp_B2 = temp_B2 / 2;
                count_2 = count_2 + 1;
            end
        end
    end
end
end
```

找 GCD 的方式我並不是用轉轉相除法，而是用因式分解的方式 (12=2\*2\*3)，透過這方法把三個數字先暫存(temp\_BX)再做因數分解，找出每個質數的最小次方數 (count\_X)，最後再拼出 GCD，這寫法很傳統也很不適合大量數

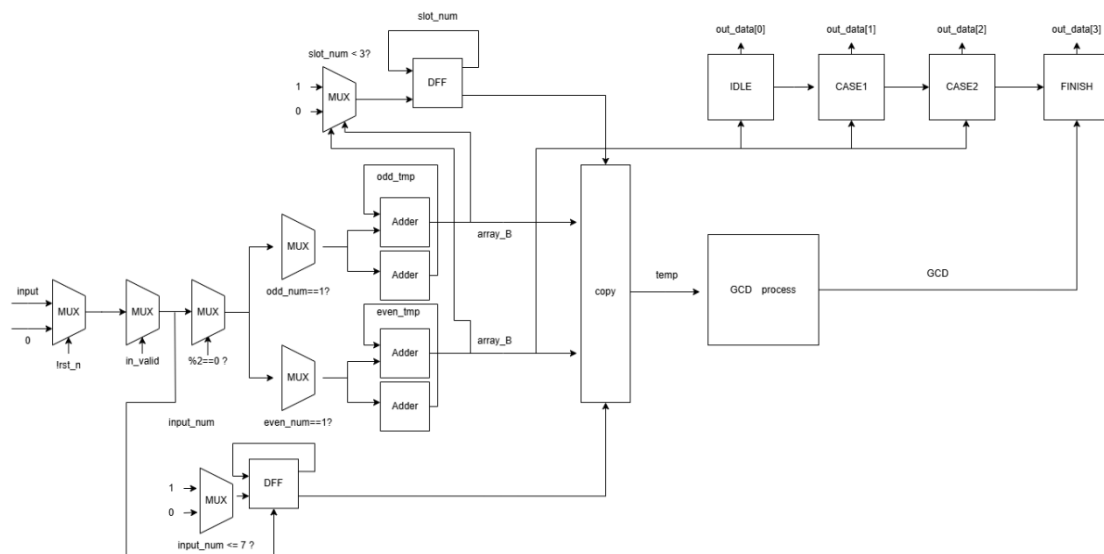
據的運算，不知道有沒有更高效的算法？

### 3. FSM 的應用

```
case (state)
  IDLE: begin
    if (slot_num == 3 && input_num >= 7) begin
      access <= 1; // 提前設置 access
      out_valid <= 1;
      out_data <= array_B[0];
      state <= CASE1;
    end
  end
  CASE1: begin
    out_valid <= 1;
    out_data <= array_B[1];
    state <= CASE2;
  end
  CASE2: begin
    out_valid <= 1;
    out_data <= array_B[2];
    state <= FINISH;
  end
  FINISH: begin
    out_valid <= 1;
    out_data <= GCD;
    refresh <= 1;
    state <= IDLE;
  end
endcase
```

我設計了四種狀態:IDLE、CASE1、CASE2、FINISH，IDLE 是設計來確認是否 input 已經輸入完了，如果是就把 out\_vaild 的電壓拉高，並輸出 array\_B[0]的值，最後更新狀態到 CASE1，CASE1 繼續拉高 out\_vaild 電壓、輸出 array\_B[1]的值並傳到 CASE2 的狀態，CASE2 的操作與 CASE1 一樣，進入到 FINISH 狀態後輸出 GCD 並且刷新所有變數 (refresh)，最後把狀態調回 IDLE。

### 4. Block diagram



### 5. 心得

這次的作業真的有困難到，在寫作業的這兩天裡遇到數不清的 bug，nWave 都不知道開了多少次，但幸好最後有寫出來，不過寫完之後發現我的想法太笨了，這樣效率真的太低了，不知道有什麼更加高效的寫法，突然來了點興趣。